

Dynamic Adaptation of Functional Runtime Systems Through External Control

Stuart Gordon
Heriot-Watt University
sg365@hw.ac.uk

Sven-Bodo Scholz
Heriot-Watt University
s.scholz@hw.ac.uk

ABSTRACT

In this paper, we present a novel approach towards providing compiler generated runtime means for dynamic adaptation. The key novelty of the proposed solution is a complete separation between the runtime adaptation mechanism itself and the control mechanism of the adaptation. This strict separation has various benefits including experimentation with adaptation control mechanisms without modifying the runtime system itself, and opening up the possibility for control mechanisms that control several applications possibly even across different runtime systems.

The paper does not only present the basic principles of the approach taken, it also describes our prototypical implementation in the context of the functional array programming language SAC. The dynamic adaptation we are concerned with in the context of this paper is an adaptation in the number of computing cores utilised during parallel execution on multiple cores. We show how this adaptation mechanism can be leveraged to deal with changes in the background load generated from other applications running concurrently with the application whose parallelism we want to control.

CCS Concepts

•**Mathematics of computing** → *Discrete mathematics*;
•**Computing methodologies** → *Parallel programming languages*; •**Software and its engineering** → *General programming languages*; *Compilers*;

Keywords

Functional Programming, SAC, Compilers, Array Programming, High-level Parallel Programming, High Performance Computing, Dynamic Adaptation, Over-subscription

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Request permissions from Permissions@acm.org.

IFL '15, September 14-16, 2015, Koblenz, Germany

© 2015 ACM. ISBN 978-1-4503-4273-5/15/09...\$15.00

DOI: <http://dx.doi.org/10.1145/2897336.2897347>

1. INTRODUCTION

As the hardware we use becomes increasingly complex, techniques for dynamic adaptation gain in importance. Even the smallest machines, such as mobile phones, today constitute an agglomeration of computing cores, graphics processors, and digital signal processing components. Running an application on such heterogeneous systems requires many non-trivial choices as to where to execute individual parts of the applications, and how to share any other given resource, such as memory, energy, or sensors of various kinds. While static planning or profiling based approaches have proven to be effective in many scenarios, they have one important restriction. They require the context of the application under consideration to be fixed. They typically do not capture availability of alternative compute capabilities, they assume that the number of processes and applications that share the system with the application under consideration is fixed and they assume that the application itself behaves in a reproducible manner. With the increasing use of virtualised shared computing infrastructure these prerequisites are less frequently given. Many applications stream big-data across a shared infrastructure and they require continuous adaptation to the resource pressures that are changing frequently.

The development of such adaptation mechanisms comes with many challenges. Three of the central questions are:

1. how can we predict resource demands;
2. how can we adapt the codes to increase or decrease the utilisation of certain resources; and
3. how should we control these adaptations?

Besides these fundamental challenges, any implementation of such adaptation mechanisms needs to be semantics preserving and it must not introduce much if any overhead. Given this plethora of challenges, a manual integration of adaptation mechanisms into individual applications seems not realistically viable. The programming effort required is far from trivial and code maintenance is likely to turn into a nightmare due to the increased complexity of the code.

We therefore believe that the only possible option is to insert such mechanisms automatically by means of advanced compilers. A prerequisite for such automated introduction of adaptation mechanisms is that the given program is suitable for this task. If we want compilers to modify an application's usage of resources such as processing power, processors used, or memory demand then the compiler needs to be able to "understand" how the application makes use of the resources. It is particularly this requirement which renders

functional programming languages excellent candidates for such automated dynamic adaptation. Their abstract and side-effect free nature typically inhibits the programmer from making explicit choices in resources and, thus, forces compilers and their runtime systems to manage resources in the first place. This gives a compiler utmost liberty for generating alternative codes that differ in the way the system resources are being utilised. Another added benefit of the state-less nature of functional programs is that dynamic changes in the executed code can be achieved rather easily. Despite these advantages of the functional setting, the task at large is still daunting. In particular the lack of answers for the three questions above combined with the non-trivial coding effort render experimentation a tedious task inhibiting quick progress.

In this paper, we try to tackle these challenges by leveraging ideas from software adaptation in software engineering [12] to integrate flexible dynamic adaptation mechanisms into applications by means of compilation. We propose to standardise and to completely externalise the actual control mechanism from the application itself. That way, experimentation with the control mechanism, i.e. finding answers to question three from above, becomes possible without requiring any modifications of the compiler, the runtime system of programs or the application itself. Given this complete separation, it is even conceivable that programmers encode their very own, application-specific controllers without any actual modification of their applications.

The particular adaptation challenge we take as an example in this paper is the degree of parallelism that is being utilised on shared memory multi-core systems. More specifically, we want to adapt the number of concurrent operating system threads that are used in order to improve the overall system performance in a setting of a dynamically changing background load on a system.

We have built a prototypical implementation in the context of the functional array language SAC [11, 15, 4]. Our implementation allows some basic performance data to be extracted from running SAC applications and provides a feedback mechanism which, in turn, regulates the parallelism of the application itself.

The individual contributions of the paper are:

- We present the design for a completely detached dynamic adaptation interface based on asynchronous 2-way communication.
- We propose a generic programming interface to facilitate the negotiation of shared resources.
- We describe our initial implementation in the context of SAC: the extensions of the runtime system, providing details of the implementation, outlining the relevant benefits of the functional setting.
- We describe a first controller for dynamic adaptation of parallelism to maximise application performance with and without background applications sharing the system.
- We present some initial performance experiences with our controller.

The paper is organised as follows. We start in Section 2 with a description of the chosen adaptation challenge which

we use as motivation throughout the paper: the effect of background applications that lead to a CPU over-subscription of the entire system. Section 3 provides the necessary background information of SAC and presents some motivating experiments that demonstrate the effects of over-subscription on SAC-programs. In Section 4 we explain the key ideas of our adaptation mechanism and we sketch the interfaces between the runtime system, the adaptation mechanism and the controller. Section 5 describes our external controller application which handles over-subscription through the interface from Section 4. Section 6 provides the details of the SAC-specific implementation before Section 7 discusses the results of our initial experiments. Section 8 presents related work before Section 9 concludes.

2. CPU OVERSUBSCRIPTION

Most parallel applications aim to fully utilise all CPUs available on the executing system. While this typically provides the best possible performance if such an application is run without other compute intensive tasks being executed on the same system at the same time, as soon as two or more such applications are run simultaneously, performance typically degrades over-proportionally. Such a situation, where more CPUs are expected than physically available, often is referred to as *oversubscription* [9].

Oversubscription has been well studied in the context of multiuser operating systems. For example, simultaneous processes can operate perfectly well when fewer threads are being utilised than freely available cores on a given system. However, runtime performance will typically suffer if oversubscription occurs, as the management and distribution of threads among competing applications becomes problematic. In particular, high-level (application) and low-level (OS) scheduling policies tend to conflict with each other during periods of oversubscription. This stems from the fact that the majority of OS scheduling mechanisms are based on the assumption that each process or thread is independent, so will be unaware of any additional scheduling mechanism. In reality there can be various levels of application-specific thread management being deployed which will operate under the assumption that all its threads will progress simultaneously until completion. The observed effect is well-known in general; as outlined by [8] there exist three prominent issues which affect current resource management techniques being applied to the parallel setting:

1. Preemption happening at inconvenient times, e.g. if a thread is scheduled out whilst holding a lock. A thread that requires the lock will stall, as its execution will not be able to proceed until the lock holder is scheduled back in. Consider a parallel loop, for example, which has had its iteration space partitioned and distributed to participating threads. If threads are preempted before synchronising, then the parallel loop cannot terminate until these threads complete.
2. A thread needs to decide whether to spin or block when it has to go into a wait state. However, a thread will typically lack the relevant information required to make the best decision (e.g., information about their environment, i.e. knowledge of any other threads executing).

3. The allocated CPU time that each thread receives can be drastically different, as it depends on whether or not the operating system has prioritised threads that have been recently unblocked.

Dealing with oversubscription in general is not easy. One possible approach is to build a central scheduling instance that is aware of all interdependencies between the individual threads of work. Several works to this effect, e.g. Callisto [7] or Lithe [13], have been proposed and their effectiveness has been proven in principle. The drawback of such a centralised solution, however, is that it only works if *all* applications that are being run simultaneously do use the centrally provided mechanisms.

In contrast, the goal of the work presented in this paper tries to provide self-adapting mechanisms that enable an application by itself to decide when to acquire more CPUs and when to release a few of them, independently of whether other applications share any information directly with the self-controlling one or not.

3. SAC AND OVERSUBSCRIPTION

SAC is a functional array programming language supporting data parallelism and code generation for high-performance executions on a wide range of many-core architectures. The key feature of the design of SAC is that all array operations provided through the standard library of SAC are mapped into a single language construct, named *with-loop*. In essence, *with-loops* are array comprehensions that combine a sophisticated form of map and reduce functionality. For details on the language and its support for compilation to many-core architectures see [15, 5, 6].

3.1 Parallel execution

The *with-loop* construct forms the basis of the parallelising compiler `sac2c`¹ which generates highly efficient multithreaded code for shared memory machines, CUDA code for GPUs, and codes for other many-core architectures. All the different back-ends of `sac2c` identify in a target-architecture-specific way which of the *with-loops* should be mapped into parallel code and which of them should be executed sequentially. The shared-memory multi-core back-end that this paper is concerned with selects coarse-granular outer-most *with-loops* only. Nested parallelism is not exploited by the runtime system. If a *with-loop* is nested and the outer with loop is parallelised, the inner with-loop will be executed sequentially by the individual threads that are responsible for the corresponding section of the outer *with-loop*.

Parallelised *with-loops* are mapped into a fork-join model of execution [17]. Each work item in the fork join model is mapped into an operating system thread, and the task-distribution is compiler generated. Currently, `sac2c` is using the API of POSIX threads, which we will refer to as pthreads in the rest of the paper. The implementation details can be found in [4].

3.2 Oversubscription

The runtime system for SAC applications is built on the assumption that all pthreads that participate in the computation of a parallel section progress in roughly the same pace. While this holds when more cores are available than

threads created, in the oversubscribed scenario this assumption breaks. We try to capture the effect of the oversubscription scenario by means of a series of runtime experiments. First, we measure the behaviour of an application when varying the number of pthreads used from 1 to the maximum of cores available without having any background load. This gives us an idea of the scaling of the chosen application. We then repeat the experiment while running a contrived background load that keep half of the existing cores busy.

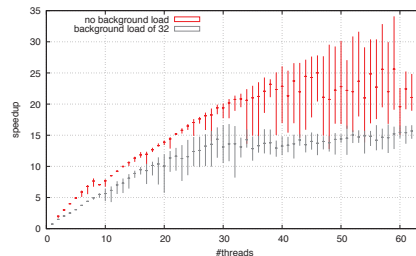


Figure 1: Effects of oversubscription when running Matrix Multiplication on 1000×1000 element matrices. With half of the 64 available cores being busy with a background application, using more than 32 cores leads to significant performance degradation.

Fig. 1 shows the effect of oversubscription observed when running matrix multiply as a 64-ways multithreaded SAC program on a 64 core machine. The graph shows speedups over sequential execution on a single core when varying the number of threads that are being used. The vertical bars for each number of threads indicate the variability of measurement, connecting the slowest with the fastest measurement from five actual measurements; the small horizontal bar indicates the mean of those measurements. The large volatility in the measurement, in particular when looking at the higher thread counts, seems to be a scheduling artefact of the operating system. Despite that volatility we can see that the application scales reasonably well if no background load is present. The second set of bars shows the performance we obtain when having a constant background load on 32 cores. Here we see rather clearly that using more than 32 threads has no significant runtime benefits. Instead, it just increases the overall demand on system resources.

4. DYNAMIC ADAPTATIONS

In this section we present our conceptual take on the problem of dynamic adaptation. Demonstrated in Fig. 1, during periods of the oversubscription an application performs much better if resource demands on the system stay within bounds of its capacity, as system load can be effectively managed. However, system loads tend to continuously change over time. In fact, one could compare this metric to a stream of some data. If constantly monitored this stream of data can be used to initiate application adaptation requests when the observed values abide to a certain criteria. For example, if the system load significantly decreased/increased we can send a message to the application to increase/decrease its resource demands.

To build such a model, we consider: where the metric is coming from, who is initiating the adaptation request, and how the application reacts to such a request. One possible way to implement this feature would be to have the application perform all these aspects by itself. As a consequence,

¹See <http://www.sac-home.org> for more details

every modification of the metric would initiate an adjustment of the whole application. Alternatively, we propose to externalise the decision mechanism out of the application into a separate entity, that we call *controller*. However, this may prove very difficult, as our assumption is that by externalising this mechanism outside of the application's process space, we may loose inherent knowledge associated to the amount of resources that the application is using now or intends to use in the future.

In order to keep the implementation lightweight and generic as possible, our approach towards runtime adaptations is based on an interface layer which enables complete separation of concerns. This approach towards dynamic adaptation of software in general is not new. In various contexts of software engineering such as code instrumentation or dynamic version updates it has been applied and often is referred to as “figure eight” model [12]. The interface layer serves as a means of releasing the stream of data produced by the application to an external controller. The controller which analyses the stream of data will in return issue a stream of adaptation requests. The “figure eight” model tailored to our specific needs is shown in Fig. 2.

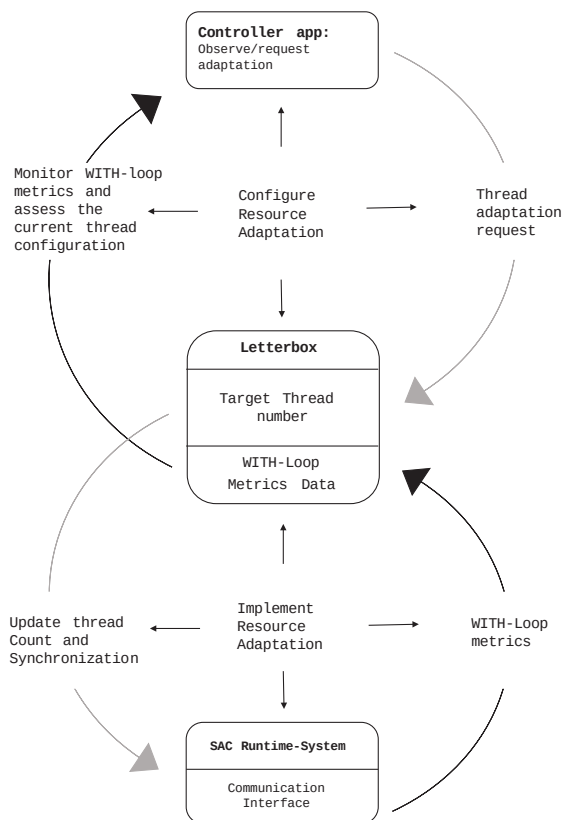


Figure 2: Dynamic Resource Adaptation

The lower part of the diagram presents the running application that generates a stream of metrics (black arrow on the right) and reacts on the adaptation requests (gray arrow on the left). The upper part of the figure is a controller; it reads the metrics (black arrow on the left) and issues adaptation requests (gray arrow on the right).

As we want to develop a compilation technology that would enrich compiled applications with the entities that

produce streams of metrics and react on adaptation requests, in principle we have to deal with a second order figure eight. The second order model would be parametrised by a set of compilation flags and in principle can instantiate multiple controlling schemes in the same applications. To keep the reasoning simpler, in this paper we are going to fix the metrics and the protocol; however, our implementation is kept as generic as possible.

For a multi-core execution, we have to choose how to implement the middle part of the figure eight in the most concise and low overhead manner. It turns out that interprocess communication via shared memory is an ideal solution. Within our implementation this entity is known as a *letterbox* as it has asynchronously operated channels with dedicated reader and writer on each end of the channel. The idea is that an application allocates a piece of shared memory. This memory serves as a drop off point where the application can output its n last values of the metric stream. The controller then asynchronously reads these values and in response may decide to issue an adaptation request by putting a message in the shared memory. Overheads of such a communication are very low and by design we have split the shared memory into application and controller parts, therefore write conflicts never happen. This means that in principle, this communication can be implemented without any expensive locking mechanisms.

While all considerations regarding the letterbox so far are generic and can be used for any kind of dynamic adaptation, the remainder of this section is concerned with the thread adjustment problem at hand. We assume that there is an application that iteratively executes the same *with-loop* in a multithreaded fashion; which is generating a stream of runtimes and the adaptation request changes the number of threads that will be allocated for further executions of the *with-loop*. This means that the interface of the letterbox from the perspective of applications can be defined as follows:

```

1 /* Allocate shared memory to observe over the
   with-loop that can be identified by 'id'. */
3 shm* allocate_shared_memory (with_loop_id id);

5 /* Register the last runtime of the with-loop
   in seconds, when using 't' threads. */
7 bool register_last_runtime (shm* s, double seconds,
   int t);

9 /* Get adaptation request. */
11 int get_new_number_of_threads (shm* s);

The interface for the controller looks as follows:

1 /* A structure to represent one item of
   a stream of metrics. */
3 struct metric {
4     int threads;
5     double time;
6 };

7 /* Attach to shared memory. */
9 bool attach_to_shared_memory (shm* s);

11 /* Get the last runtime. */
12 struct metric get_last_runtime (shm* s);

13 /* Issue adaptation request to 't'. */
15 bool set_new_number_of_threads (shm* s, int t);

```

The overall idea is to adapt the number of threads when the last n runtimes got noticeably worse than what has been observed before. The idea to measure the runtime of the *with-loop* instead of basing decisions on the system load is

motivated by the independence of the overall setup. That is, in principle the interface from above is self sufficient no matter what operating system we are using or thread scheduling policy is in place. For the time being we do not know if such an independence can be of a practical use; however from the design perspective this seems to be as minimal as it gets. Furthermore, the separation of concerns, due to the letterbox design, easily allows external information such as system loads to be integrated for generating system-specific controllers in the future.

From running applications on multi-core systems (with or without oversubscription) we know that when a number of threads allocated for a data-parallel task is close to maximum that is available on the system, the volatility of the measurements may increase dramatically. The reasons for this are manifold: cache synchronisations between the cores; scheduling artefacts; placements of threads etc. No matter what this is, our system should be able to distinguish random fluctuations from an actual increased load. We do this by defining one runtime measurement as a median of a set of measured runtimes. In order to keep the runtime overhead low, we keep this smoothing of measurements on the side of the controller. The letterbox simply provides the raw runtime data.

Finally, the adaptation itself, needs to maintain integrity of all states and state manipulations that possibly occur during the parallel execution. This is where the functional setup of SAC comes very handy. As the *with-loop* semantics guarantees a stateless construction, it can be run using any number of threads. This enables a very simple first cut at our intended adaptation: changing the number of threads between two fork-join regions can be done with minimal intrusion. In principal, the number of threads could also be changed during parallel executions. However, that would require the mapping of tasks between threads to be dynamically coordinated. For the time being, we restrict our adaptation to changes between parallel executions.

5. THE RESOURCE CONTROLLER

As explained in the previous sections, we want to control the number of threads used for *with-loop* executions. More specifically, we want to use fluctuations of previously observed runtimes as guidance for how many threads to use for the next parallel execution.

Key to this approach is the observation that a large number of the scientific codes spend most of their time in iterative approximations that apply compute-intensive, often data-parallel operations on large data structures. For this kind of code pattern, we have a very favourable situation: we run the same parallel *with-loop* over and over again. Furthermore, in most cases, the compute complexity of these loops is independent of the data. This gives rise to a very simple closed loop control model [18]: we make slight changes to the number of threads used, wait for a few iterations to be measured and then decide whether this leads to a speed-up or a slow-down. Depending on what we find we either change the number of threads used in the same direction² as before or we revert to the previous setting.

The main challenge with this approach lies in its incremental nature. Since there is no established baseline performance,

²The same direction means here that we keep increasing if we were increasing and we keep decreasing if we were decreasing.

Algorithm Thread-Controller

```

begin
  Let  $n \leftarrow \text{max\_threads}$ ;
  Let  $t_n \leftarrow \text{findQuickestTime}()$ ;
  Let  $t_1 \leftarrow t_n * n$ ;
  Let  $t_{\text{last}} \leftarrow t_n$ ;
  Let  $\text{step\_direction} \leftarrow -1$ ; decrement  $n$ ;
  Let  $\text{step\_size} \leftarrow 8$ ; amount of adjustment;
  Let  $\text{corridor\_width} \leftarrow \text{value between } [0,1]$ ;

  Control Procedure
  for ever do
     $n \leftarrow n + \text{step\_direction} * \text{step\_size}$  ;
     $\text{adjustThreads}(\text{step\_size})$  ;
     $t_n \leftarrow \text{findQuickestTime}()$  ;
    if  $t_1/t_n < (1 - \text{corridor\_width}) * n$  then
       $\text{step\_direction} \leftarrow -1$ ;
       $\text{step\_size} \leftarrow \max(n/2, 1)$ ;
    else
      if  $t_1/t_n > n$  then
         $t_1 = t_n * n$ ;
      end
      if  $t_n > t_{\text{last}}$  then
         $\text{step\_direction} \leftarrow -\text{step\_direction}$  ;
      end
       $\text{step\_size} \leftarrow \max(\text{step\_size}/2, 1)$  ;
    end
     $t_{\text{last}} \leftarrow t_n$  ;
  end

```

Algorithm 1: The variable n refers to the number of threads the controller wants to use; t_n refers to the time obtained from the runtime system.

the system at best can react against changes observed since the time it has started. Furthermore, if a fluctuation for whatever reason gets missed the system calibrates against a new, potentially wrong baseline.

The baseline of our controller is an approximation for a sequential *with-loop* execution. At the same time, we never actually want to run the *with-loop* using one thread only. The runtime of a single-threaded run can be prohibitively slow when the number of available cores is reasonably big, e.g. 64 in our experiments; running single threaded just for generating a baseline for our controller would entail an unacceptable overhead.

Assuming that we never have super-linear speedups, we approximate single threaded execution time denoted by t_1 by computing $t_n \times n$, where n is the number of threads used and t_n is the measured runtime. That way, we obtain an upper limit for t_1 which we can improve whenever a lower approximation for t_1 is found.

With such a baseline performance established we can judge for any pair of n and t_n how well we are currently scaling when using n threads. Revisiting our oversubscription experiment in Fig. 1 from Section 3, we can see that speedup can be used as a good first discriminator to decide what to do. All that is necessary is to enforce our system to stay close enough to the idealised linear scaling. To do so we define a *corridor* of acceptable speedup versus our approximation for t_1 . The overall idea is to attempt keeping the observed runtime-thread-count pairs within this corridor. If we fall

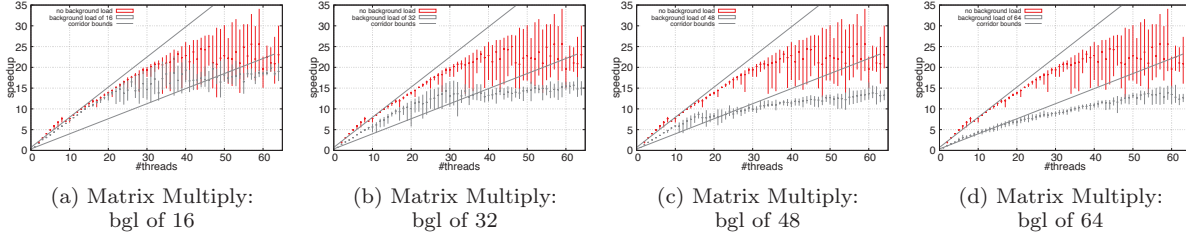


Figure 3: Introducing the notion of a *corridor* to judge the current quality of scaling with varying background loads (bgl) when running Matrix Multiplication on 1000 x 1000 element matrices

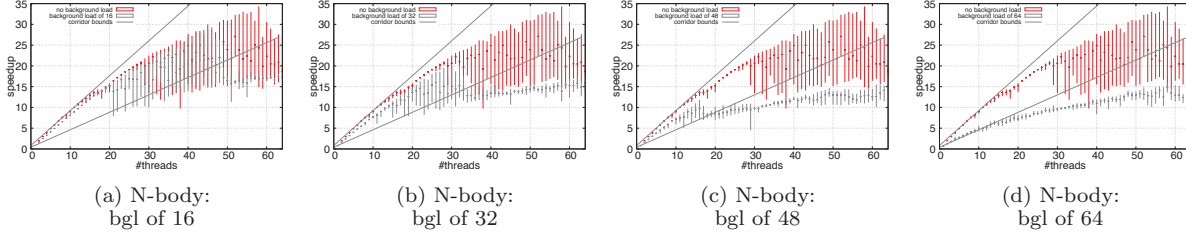


Figure 4: Introducing the notion of a *corridor* to judge the current quality of scaling with varying background loads (bgl) when running N-body on 10000 element vectors

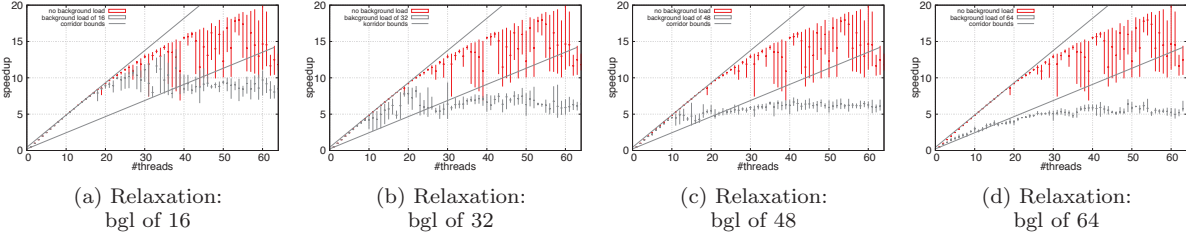


Figure 5: Introducing the notion of a *corridor* to judge the current quality of scaling with varying background loads (bgl) when running Relaxation on 10000 x 10000 matrix

below we reduce the number of threads until we get back into the corridor.

Fig. 3, Fig. 4, and Fig. 5 show the corridors for three different applications Matrix Multiply, N-body, and Relaxation, respectively. For each of these applications, we measured the effect of different background loads of 16, 32, 48, and 64 busy threads, shown in subfigures (a), (b), (c) and (d) for each figure.

Note that Fig. 3b shows the same figure as in Fig. 1 but now it additionally contains the two lines to indicate the corridor. The upper limit is defined through the best approximation for t_1 that we obtained after running the application with varying background load for about 30 minutes. The lower limit of the corridor is a fixed fraction of the upper limit. All experiments in this paper are based on 50%.

From these figures, we can already see that the corridor-based approach, at least conceptually, seems to be very apt in all these scenarios. Keeping the measurements inside the corridor should cut out the non-scaling parts of all curves.

The actual width of the corridor can be seen as a formal parameter of our controller. The “narrower” the corridor, the more sensitive the controller reacts on runtime changes. However, finding the right width of the corridor in general might be very challenging. Within all experiments investigated in

the context of this paper 50% proved suitable.

Note in Fig. 3 that a few of the measured runtimes are above the approximated ideal speedup. This is due to the fact that we obtained those runtimes by actually running on a fixed number of threads. During the run of our dynamic controller we never ran into those low thread numbers which leaves us with a slightly more conservative approximation for t_1 . If it had run with fewer cores the corridor would have been pushed higher (counter clock-wise).

Algorithm 1 shows the full controller algorithm. As mentioned earlier in Section 4, one of the first challenges in this context are measurement inaccuracies that are, inter alia, caused by OS interferences. We use a frequency distribution based technique to eliminate outliers: a fixed number of measurements (currently 20) are sorted in 5 equally spaced ranges between the fastest and the slowest execution time. The time chosen as value t_n then is the fastest runtime of the range which contains the most measurements. All this is implemented by the pseudo-function *findQuickestTime()*.

We initialise our controller by taking a first set of measurements with the maximum number of threads. The obtained runtime t_n then serves as basis for our first approximation for $t_1 = t_n * n$. The variables *step.direction* and *step.size* indicate what kind of modification for n we intend to try next.

Finally, we define the width of our corridor. The smaller we choose *corridor_width* to be, the closer our measurements need to be to the ideal speedup, i.e. the more aggressively the controller will drive the number of threads used down.

The actual closed feedback loop works as follows: After adjusting the number of threads that are being used and measuring the resulting runtime, we perform three steps:

- In case the execution time was so slow that the corresponding speedup is below our lower bound, i.e. $t_1/t_n < (1 - \text{corridor_width}) * n$, we enforce a decrease in threads by half of the current threads.
- In case the execution was so fast that the corresponding speedup was above our upper bound, i.e. $t_1/t_n > n$, we adjust our bounds accordingly by updating our approximation for t_1 .
- Unless we have dropped below our corridor, we compare the current time with the previous time. If we observed a decrease we invert the direction of thread adjustment, otherwise we keep adjusting in the same way as before. Independently of the direction of adjustment, we decrease the *step_size* to make sure we end up with adjustments of 1 thread eventually.

Due to our letterbox design as outlined in Section 4, this algorithm can be implemented as a simple C program that attaches to the shared memory of the runtime system. All interaction with the runtime system can go through the few interface functions delineated in Section 4. This enables easy experimentation even for application level programmers.

We can also see that it in principle would be easy to take system-level information or some application specific knowledge into account.

6. SAC RUNTIME SYSTEM INTEGRATION

As explained in Section 3, the source of parallelism in SAC that this paper is concerned with is non-nested fork-join-style data-parallelism implemented in pthreads. In this section, we outline how we integrate the communication with the letterbox into the runtime system of SAC. For doing so, we first sketch the code that the compiler *sac2c* generates when mapping *with-loops* to pthreads.

6.1 The Runtime System

The map/reduce nature of the semantics of *with-loops* ensures that there are no dependencies within the data a *with-loop* operates on other than a possible reduction over the results of individual tasks at the very end. This gives the compiler the freedom to choose arbitrary mappings from tasks to threads. Furthermore, threads can be run completely independently until it comes to the final synchronisation. In the runtime system, we refer to individual threads as *bees*. All bees that jointly compute a *with-loop* are being referred to as *beehive*. Each beehive has a designated *queen bee* which is responsible for coordinating the hive; the remaining bees are being referred to as worker bees.

The SAC runtime system implements an *enhanced* fork/join model [4], whereby we create only a single beehive at the start of the application and keep all bees alive till the end. The purpose of this is to avoid the overheads associated with the normal fork/join model, where threads are created and destroyed at each parallel region. Furthermore, the thread

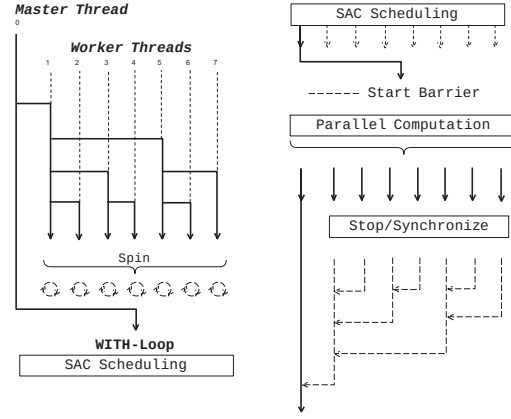


Figure 6: Outlined above is the multi-threaded execution flow of a typical SAC application. From left to right: The creation of the master thread, or Queen bee. To further tune performance, worker bees are also responsible for creating any additional worker bees by following a structured tree-like creation scheme. Once creation phase is complete bees are immediately put into a spinning state. When a *with-loop* is encountered all bees are sent to work and synchronise upon completion. Which is carried out following the same tree-like creation scheme but in reverse.

creation process is not done single-handedly by the queen bee but in a snowballing fashion, as shown in Figure 6. The queen bee creates only one worker thread, which itself creates the next worker thread and so on, till all threads have been created. In doing so, the queen bee is able to begin executing the sequential code without having to wait for all the threads to be created, allowing for the thread creation process to overlap with other computations (e.g. reading in data from a file). The thread destruction process is the same as the creation process, just in reverse.

When a multi-threaded SAC application is launched, the runtime system creates a beehive with a fixed number of threads. The number of threads is given as a command line argument or provided through an environment variable to the application. After the worker bees have been created they hit a barrier which causes them to wait for the starting point of the first parallel section. The queen continues with executing the application sequentially until it reaches a parallel region. At this point, the queen bee schedules the bees and assigns to them the parallel region code in the form of a single-program multiple-data (SPMD) frame which includes pointers to the code to be executed, pointers to the data to be used and a range of data that each bee is meant to compute over. Additionally, an output buffer is created to collect the result from the bees. Once the SPMD frame has been set up, the queen triggers the start barrier.

When the bees complete their task and return the result, they hit a synchronisation barrier at the end of the parallel region. After passing that barrier, all worker bees go into a wait state again while the queen continues with the execution of the application sequentially until we reach another parallel region. Only when the application finishes, the bees are terminated and the beehive is released.

6.2 Integration

A key aspect of the dynamic adaptation mechanism is that we are able to selectively put bees to sleep without disrupting the runtime system scheduling of SPMD frames. This is counter to the existing runtime system which uses a fixed number of bees, and schedules accordingly. In principle though, this isn't a large difference as we will continue to have the same fixed number of bees, but we put some of the bees to sleep instead of keeping them in the cycle of waiting and SPMD-frame execution, which will allow the OS to schedule these bees out. In this regard, our integration of the dynamic adaption mechanism involves (a) extending the existing bee SPMD execution mechanism to allow for putting bees to sleep, and (b) modify the runtime system scheduler to take into account the dynamically changing number of "active" bees, to ensure a consistent distribution of the computational workload.

Within a SAC application, the queen is responsible for the synchronisations of the bees, e.g. signalling the start barrier for when the bees may begin their execution. This mechanism isn't suitable though to the task of actually putting the threads to sleep, as we have different implementations for that barrier including spin-locks which keeps the thread active.

Instead, we implement a new mechanism to make the bees go asleep, by using a binary semaphore and a flag per worker bee. The flag indicates to the worker bee that it should wait on the semaphore which effectively puts the worker bee asleep. If the queen wishes to re-awake a certain sleeping worker bee, it simply resets the flag and signals on the corresponding semaphore. Once awoken, that worker bee will return to the start barrier in order to participate in the next parallel execution of the beehive. The code listing below shows the pseudo-code of the loop function executed by all worker bees:

```
1 bee_spmd_loop( sac_bee_t *self) {
    /* global barrier flag */
2     volatile unsigned * sharedfl;
    /* thread local barrier flag */
3     unsigned * locfl;
    /* worker id */
4     unsigned id = self->id;

5     for (;;) {
        sharedfl = &(self->hive->sharefl);
6         locfl = &(self->locfl);

7         /* wait on start lock: the queen will release
            * it when all is ready for an SPMD execution */
8         wait_on_barrier(locfl, sharedfl);

9         if( self->hive->bee_should_sleep[id]) {
            sem_wait( &(self->hive->sem_bee[id]));
10        } else {
            run_spmd_fun( self);
11        }
12    }
13 }
```

In line 15 we have the start barrier. It is implemented by means of one flag per bee. Each worker bee has its local flag `locfl` and the queen bee has a global flag `sharefl`. While all flags are identical all workers wait. As soon as the queen bee flips its flag, all worker bees flip their local flag as well and start executing the code after the barrier. This spinning wait and the flipping of the local flag happen in the function `wait_on_barrier`.

After passing the start barrier, each worker bee checks to see if it should go to sleep or execute its SPMD frame. If

it should sleep, it waits at the semaphore (line 18) for the queen to signal it. When eventually being woken up again the bee returns to the start barrier.

In case the worker is not ordered to go asleep, it executes the code of the SPMD frame in line 20. The code there culminates in a synchronisation of all awake bees. Thereafter, the worker bee returns to the start barrier.

In contrast, the queen bee is executing the entire program. Whenever the queen bee reaches a parallel section it executes the code that corresponds to one iteration of the worker bee loop shown below:

```
1 setup_spmd_execute( sac_bee_t *self) {
    /* global barrier flag */
2     volatile unsigned * sharedfl =
        &(self->hive->sharedfl);
3     /* Target number of bees as specified by
        * the controller */
4     unsigned target_num_bees =
        GET_TARGET_NUM_BEES();
5
6     /* calculate the time taken by the SPMD */
7     struct timespec start, end;
8
9     resize_hive( self->hive, target_num_bees);
10
11    /* start all the bees by signaling the barrier */
12    signal_barrier(sharedfl);
13
14    TIMER( start);
15
16    /* run ourself */
17    run_spmd_fun( self);
18
19    TIMER( stop);
20
21    /* update letterbox metrics */
22    UPDATE_LETTER_BOX(start, stop);
23 }
```

We can see that this code contains the main part of the direct interaction with the letterbox. First, we retrieve the target number of bees from the letterbox in line 7. Line 13 potentially reconfigures the hive, making sure that the appropriate sleep-flags are set or reset if needed. The function `resize_hive` also ensures that further administrative structures of the hive are adjusted accordingly. After resizing the hive the queen can trigger the start barrier through the code in line 16, thereby releasing the worker bees to execute their SPMD frames. In order for the resource controller to function, we do our timing measurements on lines 18 and 23. The measurement is done for the queen's execution of the SPMD frame on line 21. Finally, after our measurements are done, we pass these to the letterbox to be picked up by the resource controller.

The actual resizing of the hive is shown in the following pseudo-code:

```
1 resize_hive( sac_hive_t *hive, unsigned target_num_bees)
2 {
3     int i, temp; // temporary variables
4     int cur_num_bees = hive->current_num_bees;
5
6     if( target_num_bees <= 0) {
7         ERROR( "Trying to set hive size to 0!");
8     } else if( target_num_bees > MAX_NUM_BEES( hive)) {
9         ERROR( "Trying to set hive greater than max bees!");
10    } else {
11        /* Ensure we schedule the right number of threads */
12        SET_GLOBAL_NUM_THREADS( target_num_bees);
13
14        if( target_num_bees < cur_num_bees) {
15            for( i = cur_num_bees; i > target_num_bees; i--){
16                /* indicate that thread should SLEEP */
17                hive->bee_should_sleep[i-1] = TRUE;
```



```

19     }
20   } else {
21     for( i = cur_num_bees; i < target_num_bees; i++) {
22       /* indicate that thread should WAKE-up */
23       hive->bees_should_sleep[i] = FALSE;
24
25       /* sync threads local flag with global flag */
26       SYNC_LOCAL_GLOBAL_FLAGS( hive, i);
27
28       temp = sem_post( &(hive->sem_bee[i]));
29       if( temp < 0) {
30         ERROR( "Failed to unlock sem!");
31       }
32     }
33   }
34
35   /* Reconfigure the bee hive hierarchy */
36   RECONF_HIVE( hive, target_num_bees);
37 }

```

The function takes in the hive structure and the target number of bees variable retrieved by the queen from the letterbox. After this, we enter the stage where we might need to signal the bees - we first check that the `target_num_bees` is within sane bounds. On line 12 we call a function which sets the number of active threads that the runtime system should schedule SPMD frames for. If this is not done, the runtime system will be unable to correctly distribute the computation load across the bees. After this, we check to see if we need to signal a bee to sleep or to wake up. If the current number of active bees is greater than the target, we set the flag to trigger it to go asleep (line 17). If there are fewer bees than the target, we wake up more bees (line 22). In addition to changing the flag of the bee, we also synchronise its local barrier flag with the global one to ensure proper start barrier synchronisation. The wake up itself is accomplished by signalling the bee's semaphore (line 27).

After these stages, we finally reach line 35 where we call a function which updated the hive hierarchy. This is important, as it insures that the intercommunication/ synchronisation as shown on the right hand side in Fig. 6 is maintained.

7. EVALUATION

In this section we present some initial evaluation of the dynamic adaptation approach presented in Section 4, the controller described in Section 5, and the integration into the SAC runtime system described in the previous section. The main goals are to identify the overhead that stems from these control mechanisms and to find out how well our controller is able to adapt to different background loads.

For our evaluation, we use an 8-socket machine (8 cores per socket, each with using 2 threads per core) with AMD Opteron 6376 processors at 2.3GHz. The total main memory is 1TB and we run Scientific Linux release 6.6 (Carbon). The SAC compiler used is `sac2c 1.2.beta-BlackForest-126-31a71c8`. The C compiler used by the SAC compiler for the final code generation is `gcc 5.2.0`.

We look at three different benchmarks:

Matmul A sequential loop that performs matrix multiplication in its body,

Relaxation an iterative application of a 5-point stencil, and

Nbody a repeated application of a naive nbody simulation

This choice of mini benchmarks was driven by the attempt to cover different memory localities and different compute per memory access ratios.

7.1 Overheads

There are two main sources of overhead. First, the runtime system needs to generate the monitoring data and then push the data into the letterbox. Secondly, the runtime system needs to check for and potentially react on the control events.

The former constitutes a fixed overhead per *with-loop* execution. Its impact on the wallclock runtime therefore depends on the execution time of the parallel *with-loop* execution. We can measure this impact by comparing two runtimes: the execution without having the dynamic adaptation facility compiled in (`sac2c -mt_no_dynadapt`) and an execution after compiling with `sac2c -mt_do_dynadapt`. As long as we do not start a controller application the difference between these two times is solely made up of the monitoring of the *with-loop* runtimes and checking for potential hive resizing requests.

The overhead due to control events, i.e., the execution of `resize_hive` from the previous section and the subsequent operating system effects around the semaphores, is slightly more subtle to measure. Its impact depends on the number of resizing events that occur and potentially the number of threads that are being put asleep or woken up. Furthermore, the chosen number of threads may have an effect on the scaling and, thus, may influence the runtimes we obtain. The approach we take here is to use a machine without background load and to compare between runtimes with and without the controller. We choose problem sizes that scale reasonably well to the maximum number of threads and then start with 62 threads hoping that relatively little downscaling through the controller will happen.

The results of our overhead measurements for the three applications are shown in Fig. 7. In all three graphs, the vertical axis shows the overall runtime in seconds for 100 iterations of the respective application. On the x-axis we look at the three different versions: a version that uses a fixed number of 62 threads; a version using the dynamic adaptation interface, that starts with 62 threads and never reconfigures as we do not start the controller; and the dynamically adapting version, starting with 62 threads as well.

We found a relatively high volatility in the runtimes obtained, so we repeat all our experiments 10 times. The figures show bars between the minimum and maximum values observed and a dot for the average of all measurements taken. From the figures, we can see that the overhead due to the dynamic adapter disappears in the noise of the measurement. While there appears to be some overhead in the Matmul example, there seems to almost be an improvement in the dynamic case of N-body. Looking back at the controlled experiment in Fig. 4, we can actually see that our controller is likely to push below 60 cores, thus leading to an improved runtime due to poor scaling beyond 60 cores without background load.

7.2 Adaptation quality

We want to examine the reactivity of our controller on variations of the background load. In order to judge on the reactivity of the controller we conduct the following experiment for all three applications. We generate a synthetic background load which systematically changes the background load according to the pattern [0, 16, 32, 48, 32, 16, 16, 32, 48, 32, 16, 0], where 0 refers to no background load and 48 denotes a background load of 48 busy cores. The changes in load are made every 2 minutes, irrespective of

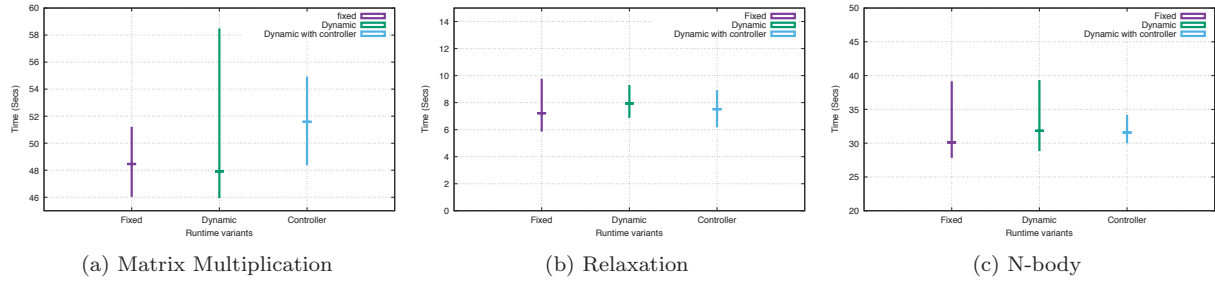


Figure 7: Overhead due to the enabling of, and deployment of, dynamic adaptations. Running Matrix Multiplication on 1000 x 1000 element matrices over 100 iterations, N-body on 10000 element vectors over 100 iteration, and Relaxation on 10000 x 10000 matrices on 100 iterations

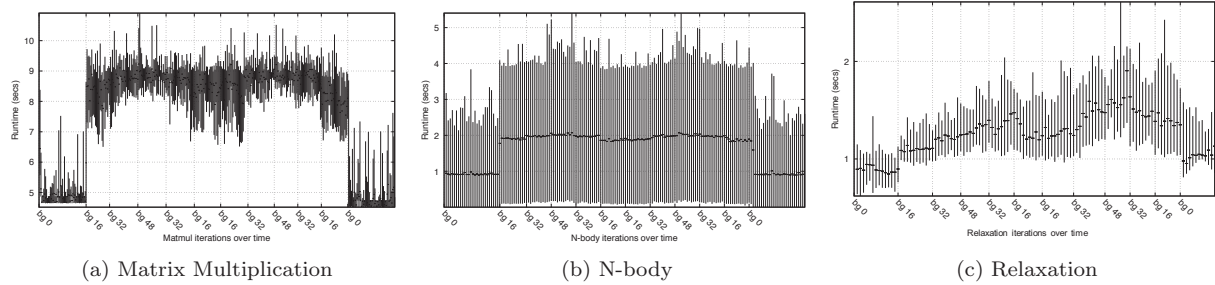


Figure 8: Running Matrix Multiplication on 1000 x 1000 element matrices, N-body on 10000 element vectors, and Relaxation on 10000 x 10000 matrices with varying background changed over 2 minute intervals. Following the following pattern of oversubscription: [16, 32, 48, 32, 16, 16, 32, 48, 32, 16]

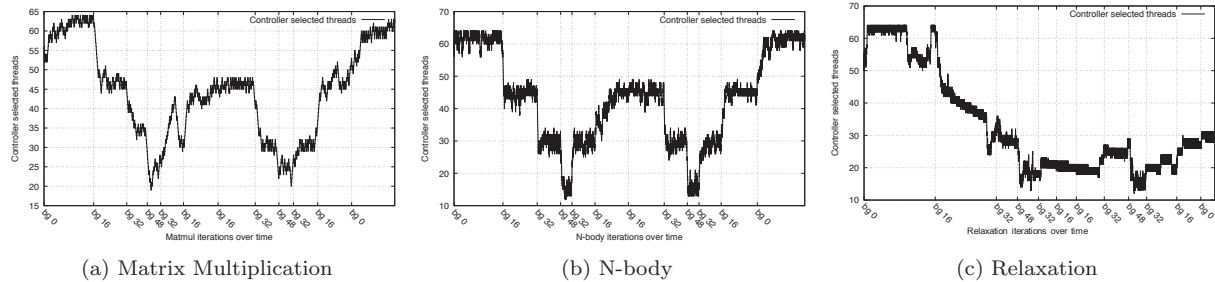


Figure 9: Running Matrix Multiplication on 1000 x 1000 element matrices, N-body on 10000 element vectors, and Relaxation on 10000 x 10000 matrices using the controller application to steer threads. The x-axis of each graph shows the total time (24 mins) till completion which is linear, but also is labeled with each experiment depicting our set of oversubscription with varying background-loads (bg) e.g. [16, 32, 48, 32, 16, 16, 32, 48, 32, 16]. The distance between each experiment is the time taken for the experiment to compute. The y-axis shows controller allocated threads

what the controller or our benchmark application does at the time.

Fig. 8 shows the individual execution times as they develop over the runtime of the entire experiment (24 minutes). The x-axis has annotations of the corresponding changes in background load as they were changed over the time of the experiment. At the example of Matmul (Fig. 8a) we can see very clearly the slowdown of a factor of 2 between background loads of 0 and 16. As the runtime roughly doubles, the number of Matmul runs that happen in the second 2 minutes of the experiment are only half as many as those that have been measured within the first 2 minutes which is

reflected in the distances between the labels on the x-axis. From these measurements, we can see that indeed the three benchmarks have rather different behaviours. Matmul reacts very pronouncedly on the existence of background load but does not vary that much between a background load of 16 and 48. N-body shows a similar overall behaviour but demonstrates much more volatility in the observed runtimes. Relaxation reacts much less pronouncedly but shows more differentiation when the level of background load changes.

Fig. 9 shows how our controller from Section 5 reacts on the controlled changes in background load for all three benchmarks. Here, the y-axis shows the number of threads

chosen by the controller while the x-axis still pertains to individual executions of the benchmarks. As in the previous figure, we have annotated the changes in background load at the x-axis to ease a correlation between background load and the behaviour of the controller.

First consider the N-body run as detailed in Fig 9b from left to right. The initial stage of the curve where there is no background load we see expected behaviour as the controller fluctuates within the range of 58-64 threads. If we consider the curve presented in Fig 4b with no background load, we can see that peak speedup is achieved using 59 threads which ties in with the controllers assumptions of 58. Also, if we consider the runtime profile in Fig 8b, runtime volatility is relatively stable. When initial background load is introduced we see a firm reaction from the controller. With the additional 16 threads active in the background we see the controller release application threads to then settle around 44-48 cores. When the background load is increased to a total of 48, we then see the controller react further by releasing more application threads in accordance with this newly introduced background load; where the thread allocations are observed to meander around their respective sweet spots. This behaviour also falls inline with our controller predictions outlined in Figures: 4a, 4b, 4c. At this point we then begin to reduce background load, first taking it down to 32 threads and then down to 16. The observed effect is again as expected, i.e. the controllers thread allocation ends up fluctuating between 44-48, although this time it has taken slightly longer to realise the ideal thread allocation whilst scaling up. At the midway section of our varying background load, we repeat the same pattern of oversubscription, which presents very similar pattern of behaviour produced by the controller. Finally, when the period of oversubscription ends we see the controller scales the thread allocation back up where it meanders between 58-64. If we compare the Matmul run as detailed in Fig. 9a against the N-body run (Fig. 9b), we see very similar curves, however due to runtime volatility (outlined in Fig. 8a), the controllers thread allocations are slightly less precise.

If we then consider the controllers adaptations in Fig 9c, and take a broader look from left to right, we see a “push down” effect occurring; were it what would seem that the controller does not recover when oversubscription ends. There are many possible reasons for this effect. When looking at Fig. 8, we see that the effects on the overall runtime for Relaxation are the least pronounced ones. Combining this with the overall volatility in measurements in the first place it is justifiable that the controller reacts more conservatively for that benchmark. Considering the average measurements without background load in Fig. 5 we can also see that beyond 30 cores there are several averages that are below the lower corridor. If such measurements strike often enough, it is not surprising to see that the controller does not raise the number of cores utilised significantly beyond 30.

8. RELATED WORK

There exists an extensive body of related work on dynamic adaptation within the software engineering community. However, that work is mainly motivated by the need for sustaining the longevity of applications rather than trying to do some dynamic performance tuning.

For example, self-adaptive software (SAS) [1] is an approach used to manage constantly changing execution en-

vironments, and the complexities which typically arise. A key idea presented in [1] outlines the need for a separation between the underlying business logic of the application and the adaptation mechanism as a way to limit complexities; due to the required integration and the expected adaptation. The motivation for the separation of the controller from the runtime system in our work is almost identical; any overhead we may incur due to analytical behaviour or decision making is completely absorbed by the controller application and has no effect on the runtime system.

J. Kramer and J. Magee [10] argue that the level of abstraction required to perform runtime adaptation stems from software architecture design. In particular, they outline Gat’s three layer robotics architecture [3] and discuss its relevancy for self actuating systems. Similar to our architecture design, Gat defines a three layers: reactive feedback control, sequencing, i.e. a reactive plan execution and deliberation. Inherent to our system is reactive feedback control, which serves as the driving force behind of our dynamic adaptation mechanism. We also incorporate reactive planning based on performance metrics (see section 5). J. Kramer and J. Magee also argue that implementing a self-managed software architecture allows not only components to interact in a way which is coherent to the overall architecture specification; but additional components and future updates will require far less explicit management.

Other related work is concerned with hard-wired dynamic adaptation. An example is the dynamic adaptation based on evolutionary algorithms used in various scenarios e.g. in tuning web-based systems [14]. They have implemented a requirements-driven controller with the aim to develop pareto-optimal solutions, i.e. each newly proposed solution must be better than or equally as good as the current in order for self-tuning to occur. Our controller uses runtime performance metrics to gauge the performance of the application. If we realise a faster runtime for a particular thread count then we update the controller with the pareto-optimal time to configure the corridor boundaries (see section 5).

Finally, there is a body of related work that tries to specifically tackle the oversubscription problem. From the perspective of the programmer, the language implementation Lithe [13] manages to provide efficient parallelism through use of a standard language interface, and runtime-system. In our implementation we also utilise a standard interface. Through abstractions we are able to offer resource management through conventional means. However, Lithe moves away from contemporary resource management by avoiding virtualised threads as to eliminating the source of common scheduling problems, such as: intrusive CPU multiplexing, a technique known to cause performance issues. Instead, Lithe offers a direct hardware abstraction, known as a hart. This is a non-virtualised hardware resources that is coordinated by the Lithe runtime-system. However, although compatible with legacy codes, and able to boast ‘bolt-on’ capabilities, Lithe can be considered as heavy weight as to be truly effective would be for Lithe to serve a unified runtime-system, requiring codes to (re)implement Lithe’s interface.

Callisto[8] is a resource management layer for parallel runtime systems envisioned and is envisioned as an experimental platform. Similar to our implementation; Callisto also implements a shared memory segment and is designed to be added as a light weight extension of a runtime-system. Callisto is able to manage resources effectively as it provides a means for

runtime systems to cooperate with each other, by enforcing work distributions policies that prevent an over subscription scenario from occurring. To the extent, that Callisto will deliberately leave one h/w context free, to prevent CPU multiplexing enforced by OS scheduling.

The ACTORS (Adaptivity and Control of Resources for Embedded Systems) [16, 2] project have developed a three tiered framework for adaptive CPU resource management by utilising: virtualisation, feedback control, and data-flow programming models. The key element of this framework is the feedback mechanism as it allows accurate assessments of CPU loads. The framework is primarily aimed at soft real-time applications where loose deadlines are specified for execution completion which is implemented using a reservation based scheduler.

9. CONCLUSIONS

This paper presents a new approach towards dynamic adaptation at the example of parallelism scaling for SAC programs. The key novelty of this approach is the strict separation between the adaptation mechanism provided by the runtime system and the adaptation control itself. We have shown that the control mechanism can even sit in a separate process without introducing noticeable overheads to the running application itself. This design opens a plethora of novel opportunities. Experiments on novel control algorithms can be tested very quickly and controllers can be exchanged while an application is running to name a few. Unless new data monitoring or new control mechanisms are needed there are no changes to the runtime system needed for such experiments at all. The strict separation also significantly simplifies experiments which take external information such as system load into account. Cooperative controlling can also be done easily by attaching to several letterboxes at the same time. This cooperation cannot only be done on the level of individual applications but also across several applications.

We also describe one particular control mechanism: the adaptation of the level of parallelism utilised on shared memory systems. Our primary goal here is to react on background system loads in order to avoid CPU oversubscription. In the paper, we show that minimal changes in the runtime system suffice to provide such adaptation functionality through our external letterbox interface. We see one major reason why this can be done with rather little effort in the fact that SAC is a declarative language which hides execution details from the programmer. If one would try to connect our letterbox to a more imperative setting where the control of resources is more explicit surely more extensive compiler technology would be required to achieve a semantics preserving tool. Even in a rather implicit setting such as openMP there would be various challenges to overcome. OpenMP's flexibility in how the parallelism can be expressed alone would require extensive program analysis to guarantee semantics preservation.

Finally we also present a very simple controller for steering the parallelism adaptation in certain classes of applications. Solely based on runtime samples, the dynamic controller regulates the parallelism towards improved runtimes. Our initial experiments on a few SAC examples show that even this very simple, locally operating control mechanism has the potential to react on global resource sharing. In several experiments the controller successfully manages to react on external oversubscription even without these applications

providing any controller interface and without any information from the operating system. The very same controller also exhibits another interesting property: *with-loops* with poor scaling are brought to voluntarily releasing thread resources to the system, thus enabling a better overall system utilisation.

Clearly, the proposed controller is not universally effective. First of all, it requires a particular program pattern: an iterative kernel that applies the same data-parallel operation over and over again. While this can be found in many compute intensive applications it is not present in all programs. Secondly, the data-parallel operation needs to be compute intensive enough to deliver sufficiently large runtimes as otherwise a high-level of volatility in the measurements impede the effectiveness of the controller. Again, this typically is a given in scientific applications but for some applications just does not hold true. Finally, the number of iterations needs to be high enough to allow the controller to react. Our experiments have shown that roughly 50-100 iterations are needed for the controller to react to a major change such as 32 cores in background load. Assuming a scenario where applications run over several hours on a compute server even this requirement is actually often met. Background loads only change infrequently on such systems and iterations often run several thousands of instances.

Given the good results in the particular scenario we have looked at we are convinced that the presented approach creates a suitable platform for broadening the applicability of our dynamic adaptation mechanism. To start with, the utilisation of external information would surely significantly widen the applicability of the proposed controller for dealing with oversubscription. Furthermore, the proposed approach lends itself towards an extension into more advanced runtime adaptations such as thread migration in heterogeneous systems or even time-cost trade-offs in commercial cloud systems.

10. ACKNOWLEDGEMENTS

We would like to thank the reviewers for the very helpful suggestions and pointers to related work. We also would like to thank the other lab members at Heriot-Watt for their input and support, most prominently Artjoms Šinkarovs and Hans-Nikolai Vießmann. This work was supported in part by grant EP/L00058X/1 from the UK Engineering and Physical Sciences Research Council (EPSRC).

11. REFERENCES

- [1] M. Amoui, M. Derakhshanmanesh, J. Ebert, and L. Tahvildari. Achieving dynamic adaptation via management and interpretation of runtime models. *Journal of Systems and Software*, 85(12):2720 – 2737, 2012. Self-Adaptive Systems.
- [2] K.-E. Arzen, V. Romero Segovia, M. Kralmark, S. Schorr, A. Meher, and G. Fohler. Actors adaptive resource management demo. In *3rd Workshop on Adaptive and Reconfigurable Embedded Systems*, 2011.
- [3] E. Gat. Three-layer architectures, artificial intelligence and mobile robots. *MIT/AAAI Press*, 1997.
- [4] C. Grelck. Shared memory multiprocessor support for functional array processing in sac. *J. Funct. Program.*, 15(3):353–401, May 2005.

- [5] C. Grelck and S.-B. Scholz. SAC: A functional array language for efficient multithreaded execution. *International Journal of Parallel Programming*, 34(4):383–427, 2006.
- [6] J. Guo, J. Thiagalingam, and S.-B. Scholz. Towards Compiling SaC to CUDA. In Z. Horváth and Viktória Zsók, editors, *10th Symposium on Trends in Functional Programming (TFP'09)*, pages 33–49. Intellect, 2009.
- [7] T. Harris and S. Kaestle. Callisto-rtts: Fine-grain parallel loops. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 45–56, Santa Clara, CA, July 2015. USENIX Association.
- [8] T. Harris, M. Maas, and V. J. Marathe. Callisto: Co-scheduling parallel runtime systems. In *Proceedings of the Ninth European Conference on Computer Systems, EuroSys '14*, pages 24:1–24:14, New York, NY, USA, 2014. ACM.
- [9] C. Iancu, S. Hofmeyr, F. Blagojevic, and Y. Zheng. Oversubscription on multicore processors. In *Parallel Distributed Processing (IPDPS), 2010 IEEE International Symposium on*, pages 1–11, April 2010.
- [10] J. Kramer and J. Magee. Self-managed systems: an architectural challenge. In *Future of Software Engineering, 2007. FOSE '07*, pages 259–268, May 2007.
- [11] D. Kreye. *A Compiler Backend for Generic Array Programming*. PhD thesis, University of Kiel, 2003.
- [12] P. Oreizy, N. Medvidovic, and R. N. Taylor. Runtime software adaptation: Framework, approaches, and styles. In *Companion of the 30th International Conference on Software Engineering, ICSE Companion '08*, pages 899–910, New York, NY, USA, 2008. ACM.
- [13] H. Pan, B. Hindman, and K. Asanović. Composing parallel software efficiently with lithe. *SIGPLAN Not.*, 45(6):376–387, June 2010.
- [14] X. Peng, B. Chen, Y. Yu, and W. Zhao. Self-tuning of software systems through dynamic quality tradeoff and value-based feedback control loop. *Journal of Systems and Software*, 85(12):2707 – 2719, 2012. Self-Adaptive Systems.
- [15] S.-B. Scholz. Single assignment c – efficient support for high-level array operations in a functional setting. In *Journal of Functional Programming*, pages 1005–1059. Cambridge University Press, 2003.
- [16] V. R. Segovia. *Adaptive CPU resource management for multicore platforms*. PhD thesis, Department of Automatic Control, Lund University, 2011.
- [17] A. Thomasian. Analysis of fork/join and related queueing systems. *ACM Comput. Surv.*, 47(2):17:1–17:71, Aug. 2014.
- [18] X. Zhu, M. Uysal, Z. Wang, S. Singhal, A. Merchant, P. Padala, and K. Shin. What does control theory bring to systems research? *SIGOPS Oper. Syst. Rev.*, 43(1):62–69, jan 2009.